JARVIS ENVIRONMENT SYSTEM LAYOUT

System Classification: Repository Configuration & Documentation Tracking Matrix
This blueprint establishes the exact multi-folder repository organization and sets the mandatory structure for every module added to JARVIS.

I. Repository Directory Tree

```
my-jarvis/
├── .github/                    # Global automation, issue templates,
and project board states
├── docs/                       # Global system blueprints, reference
architectures, and ledger
|   ├── ARCHITECTURE_BLUEPRINT.md
|   └── REPO_LAYOUT.md          # This structural definition file
|
└── src/                        # The Core Systems Layer
    ├── intelligence/           # Brain, parsing, and planning logic
    ├── memory/                 # Context windows, vector databases,
long-term semantic engines
    ├── vision/                 # Ambient parsing, desktop scanning,
and OCR frames
    ├── communication/          # I/O handling, unified audio hooks,
and keyboard listener loops
    ├── automation/             # OS-level execution macros and
workspace configuration tools
    ├── security/               # Sandboxing, safe vulnerability
scripting, and code analysis
    |   └── [module_name]/      # Example: local_sandbox_analyzer/
    |       ├── docs/
    |       |   ├── 01_definition.md
    |       |   └── 02_design.md
    |       ├── src/
```

```
    |       |   └── core_logic.py
    |       ├── iterations/
    |       |   ├── v1_basic_script/
    |       |   └── v2_oop_refactor/
    |       ├── tests/
    |       |   └── test_plan.md
    |       └── review.md
    |
    └── personality/           # Behavioral matrices, behavioral
tuning profiles, and state controls
```

II. Mandatory Module Blueprint (The 5 Deliverables)

Every functional component or application sequence created within any of the 7 pillars must reside inside its own self-contained directory containing exactly these five structural requirements:

1. The Functional Definition (docs/01_definition.md)

This file establishes the core problem space before engineering starts. It must include:

- **The Vision:** A clear statement defining perfect execution for this module.
- **Problem Statement:** A brief analysis of the specific technical limitation or gap being eliminated.
- **Sub-Module Task Assignments:** A breakdown chart mapping the localized code tasks required.
- **The Exception Map:** A predictive structure detailing how the module intercepts crashes, syntax failures, or unexpected environments cleanly.

2. The Architectural Design (docs/02_design.md)

This file serves as the strict engineering contract. It must include:

- **Pillar Justification:** Statement proving exactly why and where this fits within the 7 pillars.
- **Data Flow Diagram (DFD):** A clear text-based map or diagram showing data ingestion, transformations, storage locations, and outputs.
- **The Technology Stack:** Explicit definition of languages (C vs. Python), local library frameworks, and third-party tools selected.
- **Risk Analysis:** A listing of potential catastrophic runtime or compile-time failures, along

with predefined fallback mitigation plans.

3. Code & Multi-Iteration Management (src/ & iterations/)

- **Active Logic File (src/):** Houses the active, production-ready source code currently in use by the system layer.
- **Architectural Iteration Vault (iterations/):** Contains dedicated nested folders (e.g., v1_prototype/, v2_oop_refactor/) to store code snapshots from previous builds, protecting older functional states for historical rollback.

4. The Verification Strategy (tests/test_plan.md)

A dedicated automated file or procedural verification log containing:

- **Edge-Case Vectors:** Detailed parameters to test code limits against network drops, resource starvation, or corrupted variables.
- **Staging Verification Setup:** Specific local environmental instructions to execute isolation runs before merging any updates into production modules.

5. Post-Mortem & Reflection Ledger (review.md)

The technical conclusion document recorded after production deployment:

- **Performance Metrics:** Actual verification of constraints (e.g., execution latencies, resource footprint, vector search accuracy).
- **Architect Review:** Retrospective logging of lessons learned, hidden code logic debts to pay later, and future optimizations.